

The solution key includes all the extra questions not included in the EEE4773 midterm exam.

1. (10 points) Consider a binary classification task where each sample $\mathbf{x}_i$ is d -dimensional with its corresponding label $t_i \in \{-1, 1\}$. Suppose you have a dataset of N i.i.d. samples, $\{(x_i, t_i)\}_{i=1}^N$. Let $\mathbf{m}_k$ be the sample average for class k , and $\mathbf{m}$ the overall sample average. Suppose that your goal is to train a discriminant function of the form $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$ that separates the two classes.

Suppose that you are interested in finding $\mathbf{w}$ and w_0 such that the *projected between-class variance is maximized*. Write down the objective function, the Lagrangian function, and derive for the analytical solutions for $\mathbf{w}$ (not w_0).

For a dataset with N samples and d features, let's assume there are K classes. We can define the between-class Variance for class k , $k = 1, 2, \dots, K$ as:

$$\mathbf{S}_b^{(k)} = (m_k - m)^2 = \mathbf{w}^T (\mathbf{m}_k - \mathbf{m})^T (\mathbf{m}_k - \mathbf{m}) \mathbf{w} = \mathbf{w}^T \mathbf{S}_v^{(k)} \mathbf{w}$$

where $\mathbf{S}_v^{(k)} = (\mathbf{m}_k - \mathbf{m})^T (\mathbf{m}_k - \mathbf{m})$, m_k is the projected sample average for class C_k , the $\mathbf{m}_k$ is the original sample average, m is the overall projected sample average, and $\mathbf{m}$ is the overall original sample average. For K classes, the total between-class variance can be defined as:

$$\mathbf{S}_b = \frac{1}{K} \sum_{k=1}^K \mathbf{S}_b^{(k)} = \frac{1}{K} \sum_{k=1}^K \mathbf{w}^T \mathbf{S}_v^{(k)} \mathbf{w} = \frac{1}{K} \mathbf{w}^T \left(\sum_{k=1}^K \mathbf{S}_v^{(k)} \right) \mathbf{w} = \frac{1}{K} \mathbf{w}^T \mathbf{S}_v \mathbf{w}$$

The objective function can be written as:

$$\begin{aligned} & \arg_{\mathbf{w}} \max \frac{1}{K} \mathbf{w}^T \mathbf{S}_v \mathbf{w} \\ & \text{subject to } \mathbf{w}^T \mathbf{w} = 1 \end{aligned}$$

The Lagrangian function is:

$$\mathcal{L}(\mathbf{w}, \lambda) = \frac{1}{K} \mathbf{w}^T \mathbf{S}_v \mathbf{w} + \lambda(1 - \mathbf{w}^T \mathbf{w})$$

Deriving with respect to $\mathbf{w}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \iff \frac{1}{K} \frac{1}{K} \mathbf{S}_v \mathbf{w} = \lambda \mathbf{w}$$

Thus, the solution for $\mathbf{w}$ is the eigenvector $\frac{1}{K} \mathbf{S}_v$ with the **largest** eigenvalue.

2. (10 points) **Consider a support vector machine (SVM) classifier, $y(x) = \mathbf{w}^T \phi(x) + w_0$, for a two-class problem, $t \in \{-1, 1\}$, with the objective function,**

$$\arg_{\mathbf{w}} \min \left\{ \frac{1}{2} \|\mathbf{w}\|^2 \right\} \quad \text{subject to} \quad t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0) \geq 1 \quad \forall n$$

Explain how does this objective function maximizes the margin of the discriminant function and avoids misclassified points.

The training samples $\mathbf{x}_n$ that are correctly classified, will have so that the sign of its target label, $t_n \in \{-1, 1\}$ and the sign of the discriminant function, $y(\mathbf{x}_n)$, are the same. That is, all correctly classified points $\mathbf{x}_n$ satisfy:

$$t_n y(\mathbf{x}_n) = t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0) > 0$$

The displacement of point $\mathbf{x}_n$ to the discriminant function is defined as $\frac{y(\mathbf{x}_n)}{\|\mathbf{w}\|}$. This quantity will be positive or negative, depending if $y(\mathbf{x}_n)$ is positive (region for deciding class 1), or negative (region for deciding class -1). We can write this as a distance, but ensuring it's ≥ 0 . So, all correctly classified points, will have the following distance to the discriminant function:

$$t_n \frac{y(\mathbf{x}_n)}{\|\mathbf{w}\|} = \frac{t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0)}{\|\mathbf{w}\|} > 0$$

The points that are closest to the discriminant function, the support vectors, will have a distance, δ to the discriminant function, thus, we can write: $t_n \frac{y(\mathbf{x}_n)}{\|\mathbf{w}\|} = \frac{t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0)}{\|\mathbf{w}\|} \geq \delta$.

If we normalize the vector $\mathbf{w}$ so that this distance is ≥ 1 , we find:

$$t_n \frac{y(\mathbf{x}_n)}{\|\mathbf{w}\|} = \frac{t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0)}{\|\mathbf{w}\|} \geq 1$$

So, we can write the optimization problem as:

$$\arg_{\mathbf{w}, w_0} \max \left[\frac{1}{\|\mathbf{w}\|} \min_n (t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0)) \right]$$

We can rewrite this problem as a minimization problem:

$$\arg_{\mathbf{w}} \min \left\{ \frac{1}{2} \|\mathbf{w}\|^2 \right\} \quad \text{subject to} \quad t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + w_0) \geq 1 \quad \forall n \quad \blacksquare$$

3. (10 points) Consider the following 2-dimensional dataset, (x_1, x_2) , and its respective ground truth label, t :

Consider the Perceptron classifier with the initial weights $w^{(0)} = [4, 1]$ and bias $w_0^{(0)} = 2$. Answer the following questions:

x_1	x_2	t
1	0	1
4	2	1
0	-1	-1
-1	-1	-1
-2	1	-1

- (a) (3 points) Using the stochastic gradient descent, with a learning rate of $\eta = 0.1$, compute the new weights $w^{(1)}$ and the new bias $w_0^{(1)}$. The discriminant function can be written as $y(\mathbf{x}) = \phi(\mathbf{w}^T \mathbf{x} + w_0) = \phi(4x_1 + x_2 + 2)$ where $\phi(x) = \begin{cases} 1, & x > 0 \\ -1, & x \leq 0 \end{cases}$ is the threshold function. We classify as 1 if $y(\mathbf{x}) > 0 \iff 4x_1 + x_2 + 2 > 0$, otherwise classify as 0. We can rewrite the discriminant function as: $x_2 = -4x_1 - 2$.

The output predictions, y , for each of the 5 data samples are as follows:

x_1	x_2	t	y
1	0	1	1
4	2	1	1
0	-1	-1	1
-1	-1	-1	-1
-2	1	-1	-1

As the Perceptron algorithm uses misclassified points to make updates in its parameters (weights and bias), the first step is to identify if there are any misclassified points.

For the table above, we can see that data sample $(0, -1)$ is misclassified. So this will be the only point that will be used to make the updates for the weights and bias term of the Perceptron.

The update equations are given as:

$$\begin{aligned} \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)} \\ \text{where } \Delta \mathbf{w}^{(t)} &= -\eta \nabla J(\mathbf{w}^{(t)}), \text{ and} \\ w_0^{(t+1)} &= w_0^{(t)} + \Delta w_0^{(t)} \\ \text{where } \Delta w_0^{(t)} &= -\eta \nabla J(w_0^{(t)}) \end{aligned}$$

Thus,

$$\begin{aligned}\mathbf{w}^{(1)} &= \mathbf{w}^{(0)} + \Delta \mathbf{w}^{(0)} = \mathbf{w}^{(0)} + \eta \mathbf{x}_n t_n = \begin{bmatrix} 4 \\ 1 \end{bmatrix} + \eta \begin{bmatrix} 0 \\ -1 \end{bmatrix} (-1) = \begin{bmatrix} 4 \\ 1 + \eta \end{bmatrix} \Rightarrow \mathbf{w}^{(1)} = \begin{bmatrix} 4 \\ 1.1 \end{bmatrix} \\ w_0^{(1)} &= w_0^{(0)} + \Delta w_0^{(0)} = w_0^{(0)} + \eta t_n = 2 + \eta(-1) = 2 - \eta \Rightarrow w_0^{(1)} = 1.9\end{aligned}$$

- (b) (3 points) **Compute another update for the weights $w^{(2)}$ and the bias $w_0^{(2)}$.** We first need to recompute the output for each sample to determine the new set of misclassified samples. We will find that sample $(0, -1)$ is still (the only) misclassified. Using the equations from above, we find:

$$\begin{aligned}\mathbf{w}^{(2)} &= \mathbf{w}^{(1)} + \eta \mathbf{x}_n t_n = \begin{bmatrix} 4 \\ 1.1 \end{bmatrix} + \eta \begin{bmatrix} 0 \\ -1 \end{bmatrix} (-1) = \begin{bmatrix} 4 \\ 1.1 + \eta \end{bmatrix} \Rightarrow \mathbf{w}^{(2)} = \begin{bmatrix} 4 \\ 1.2 \end{bmatrix} \\ w_0^{(2)} &= w_0^{(1)} + \eta t_n = 1.9 + \eta(-1) = 1.9 - \eta \Rightarrow w_0^{(2)} = 1.8\end{aligned}$$

- (c) (4 points) **Repeat part (b) but using a momentum term with $\alpha = 0.9$.** The update equations with a standard momentum term are given as:

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \Delta \mathbf{w}^{(t)} \\ \text{where } \Delta \mathbf{w}^{(t)} &= -\eta \nabla J(\mathbf{w}^{(t)}) + \Delta \mathbf{w}^{(t-1)}, \text{ and} \\ w_0^{(t+1)} &= w_0^{(t)} + \Delta w_0^{(t)} \\ \text{where } \Delta w_0^{(t)} &= -\eta \nabla J(w_0^{(t)}) + \Delta w_0^{(t-1)}\end{aligned}$$

At iteration $t = 2$, we have a previous gradient update that can be used in the momentum update. The gradient update is

$$\begin{aligned}\Delta \mathbf{w}^{(2)} &= -\eta \nabla J(\mathbf{w}^{(1)}) + \alpha \Delta \mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} + 0.9 \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.19 \end{bmatrix} \\ \Delta w_0^{(2)} &= -\eta \nabla J(w_0^{(1)}) + \alpha \Delta w_0^{(0)} = -0.1 - 0.1 \times 0.9 = -0.19\end{aligned}$$

Thus,

$$\begin{aligned}\mathbf{w}^{(2)} &= \begin{bmatrix} 4 \\ 1.1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.19 \end{bmatrix} = \begin{bmatrix} 4 \\ 1.29 \end{bmatrix} \\ w_0^{(2)} &= 1.9 - 0.19 = 1.71\end{aligned}$$

Notice that the weights and bias move faster when compared to part (b).

4. (10 points) Suppose you have a training dataset with 10 samples $\{x_1, x_2, \dots, x_{10}\}$ and the associated target labels:

$$t = \{1, 2, 2, 3, 1, 2, 3, 3, 2, 1\}$$

Suppose you trained an MLP to produce a confidence of target given a sample. For the above data points, the binary classifier produced the following confidence values:

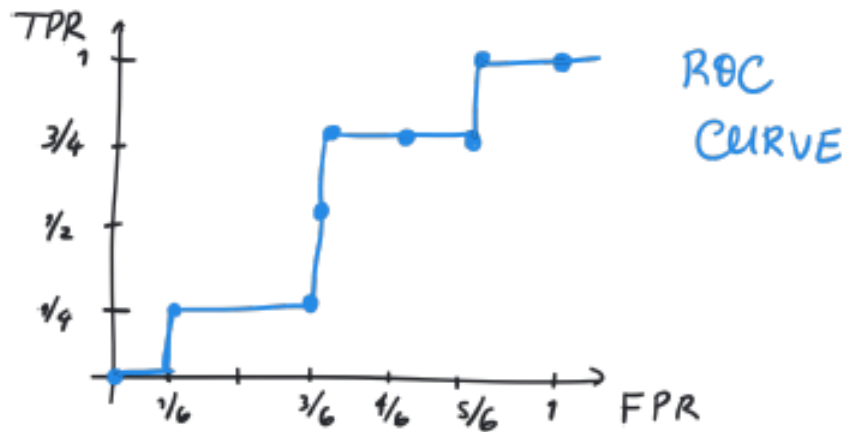
$$c = \{0.7, 0.6, 0.2, 0.3, 0, 0.9, 0.7, 0.4, 0.5, 0.99\}$$

Answer the following questions:

- (a) (6 points) Draw the associated ROC curve for class 2. Show your work.

Label y	Confidence c	Binary Label	Threshold	TPR	FPR
1	0	0	≥ 0	1	1
2	0.2	1	≥ 0.2	1	5/6
3	0.3	0	≥ 0.3	3/4	5/6
3	0.4	0	≥ 0.4	3/4	4/6
2	0.5	1	≥ 0.5	3/4	3/6
2	0.6	1	≥ 0.6	2/4	3/6
1	0.7	0	≥ 0.7	1/4	3/6
3	0.7	0	≥ 0.8	1/4	1/6
2	0.9	1	≥ 0.9	1/4	1/6
1	0.99	0	≥ 1	0	0

The resulting ROC curve is as follows:



- (b) (4 points) Suppose that the point of operation is a probability of 0.5, that is, a probability ≥ 0.5 is assigned to positive class (class 2), and probability < 0.5 is assigned to negative class (not class 2). What is the resulting confusion matrix? Show your work.

For a threshold $\delta \geq 0.5$, the resulting confusion matrix for class 2 is as follows:

		Predicted labels	
		Class 2	Not class 2
True labels	Class 2	TP = 3	FN = 1
	Not class 2	FP = 3	TN = 3

5. (10 points) **Consider a feature matrix X of shape $D \times N$, where D is the dimensionality and N is the sample size (assume $N > D$). Assume that the mean of each feature is 0, that is, $\mu_X = 0$. How is the covariance matrix of X utilized in PCA? What is the significance of eigenvalues and eigenvectors in PCA? How are they related to the principal components?**

Solve for the principal components analytically to answer these questions.

The PCA projection is

$$Y = AX$$

where A is a $D \times D$ matrix, X is a $D \times N$ demeaned data matrix and therefore Y is also a $D \times N$ transformed data matrix. The (co)variance of the transformed data Y is given by:

$$\begin{aligned}
 R_Y &= E[YY^T] \\
 &= E[AX(AX)^T] \\
 &= E[AXX^T A^T] \\
 &= AE[XX^T]A^T \\
 &= AR_X A^T
 \end{aligned}$$

Similarly, R_X represents the covariance of the data X . If we write A in terms of vector elements (assuming only 2 of them):

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} \vec{a}_1^T \\ \vec{a}_2^T \end{bmatrix}$$

Then,

$$\begin{aligned}
 R_Y &= \begin{bmatrix} a_1^T \\ a_2^T \end{bmatrix} R_X \begin{bmatrix} a_1 & a_2 \end{bmatrix} \\
 &= \begin{bmatrix} a_1^T R_X a_1 & a_1^T R_X a_2 \\ a_2^T R_X a_1 & a_2^T R_X a_2 \end{bmatrix}
 \end{aligned}$$

Consider the case where we are trying to project the data X into a 1-dimensional space, so we are trying to find the direction a_1 where maximal data variance is preserved:

$$\arg_{a_1} \max a_1^T R_X a_1$$

We want this solution to be bounded (considering $a_1 = \infty$ would maximize), so we need to constraint the vector to have norm 1

$$\|a_1\|_2^2 = 1 \iff a_1^T a_1 = 1 \iff a_1^T a_1 - 1 = 0$$

Then, we using Lagrange Optimization:

$$\mathcal{L}(a_1, \lambda_1) = a_1^T R_X a_1 - \lambda_1 (a_1^T a_1 - 1)$$

Solving for a_1 :

$$\frac{\partial \mathcal{L}}{\partial a_1} = 0 \iff 2R_X a_1 - 2\lambda_1 a_1 = 0 \iff R_X a_1 = \lambda_1 a_1$$

Thus a_1 is the eigenvector of R_X associated with the largest eigenvector λ_1 . a_1 is known as a principal component.

6. (15 points) **This problem addresses Principal Component Analysis (PCA). Answer the following questions:**

(a) (5 points) **How do you interpret the variance explained by each principal component?**

In PCA, the variance explained by each component reveals its importance in capturing patterns in the data. PC1 holds the most variance, while subsequent components hold less, all being orthogonal. Higher variance in a component indicates it retains more information from the original data, aiding in understanding its structure.

(b) (5 points) **What are the limitations of PCA? Discuss scenarios where PCA might not be suitable for dimensionality reduction.**

Some of the limitations of PCA are as follows:

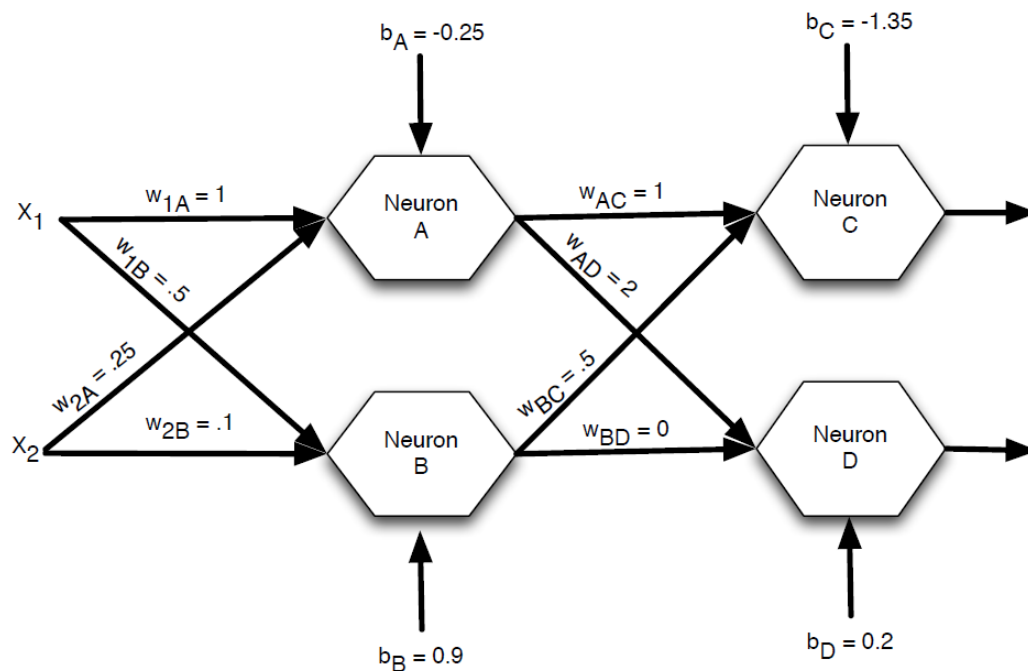
- **Linear Transformation.** In scenarios where the underlying data structure is highly nonlinear, PCA may not provide optimal dimensionality reduction.
- **Loss of Interpretability.** PCA features are linear combinations of the original features, the interpretability of these components might be challenging.
- **Preservation of Small Variance Components.** PCA prioritizes components with high variance, potentially leading to the loss of information from components with small variances. In scenarios where these small variance components are crucial for understanding the data, PCA might not be suitable.

- Assumption of Orthogonality. PCA assumes that the principal components are orthogonal to each other. In real-world datasets, this assumption may not hold true, especially if there are correlations between features that PCA fails to capture adequately.
- Scaling Dependence. PCA is sensitive to the scale of features.
- Loss of Discriminative Information. PCA focuses on capturing variance in the data but does not consider class labels or discriminative information. In classification tasks, preserving class-related information is essential, and PCA might not be the best choice for dimensionality reduction in such cases.

(c) (5 points) **How does PCA perform in the presence of multicollinearity among features? Does it mitigate or exacerbate the issue?**

Multicollinearity occurs when two or more features in a dataset are highly correlated. PCA works by transforming the original features into a new set of orthogonal (uncorrelated) principal components. In the process, PCA identifies and captures the directions of maximum variance in the data. When features are highly correlated (multicollinearity), PCA tends to group them together into a smaller number of principal components, effectively reducing the dimensionality of the dataset. Thus, PCA can help alleviate multicollinearity by transforming the original features into orthogonal principal components and reducing the dimensionality of the dataset.

7. (15 points) **Consider the following multi-layer perceptron (MLP) with 2 input units, 1 hidden layer with 2 units and 1 output layer with 2 units: All**



activation functions are the ReLU activation function, $\phi(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$ and

$$\phi'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}.$$

Consider the following objective function,

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N (t_i - y_i)^2$$

Consider the input sample $\mathbf{x} = [2, 1]^T$, with target $\mathbf{t} = [1.5, 4]^T$. Answer the following questions:

- (a) (3 points) **Carry one forward pass for this training sample.**

The output for each unit in the network can be found with:

$$y_A = \phi \left(x_1 + \frac{1}{4}x_2 - \frac{1}{4} \right) = \phi(2) = 2$$

$$y_B = \phi \left(\frac{1}{2}x_1 + \frac{1}{10}x_2 + \frac{9}{10} \right) = \phi(2) = 2$$

$$y_C = \phi \left(y_A + \frac{1}{2}y_B - 1.35 \right) = \phi(1.65) = 1.65$$

$$y_D = \phi(2y_A + 0.2) = \phi(4.2) = 4.2$$

The error in the output layer is: $\mathbf{e} = \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} = \begin{bmatrix} t_1 - y_C \\ t_2 - y_D \end{bmatrix} = \begin{bmatrix} -0.15 \\ -0.2 \end{bmatrix}$.

- (b) (6 points) **Use backpropagation with standard gradient descent and a learning rate of $\eta = 0.01$ to update the weights w_{AC} and w_{AD} . Show your work.**

The update for w_{AC} is:

$$\begin{aligned} w_{AC} &= 1 - \eta \frac{\partial J}{\partial e_1} \frac{\partial e_1}{\partial y_C} \frac{\partial y_C}{\partial v_C} \frac{\partial v_C}{\partial w_{AC}} \\ &= 1 - 0.01 \times e_1 \times (-1) \times \phi'(v_C) \times y_A \\ &= 1 + 0.01 \times (-0.15) \times \phi'(2.15) \times y_A \\ &= 1 + 0.01 \times (-0.15) \times 2 \\ &= 0.997 \end{aligned}$$

The update for w_{AD} is:

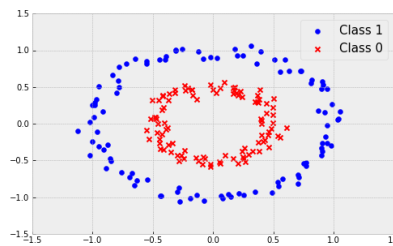
$$\begin{aligned} w_{AD} &= 1 - \eta \frac{\partial J}{\partial e_2} \frac{\partial e_2}{\partial y_D} \frac{\partial y_D}{\partial v_D} \frac{\partial v_D}{\partial w_{AD}} \\ &= 2 - 0.01 \times e_2 \times (-1) \times \phi'(v_D) \times y_A \\ &= 2 + 0.01 \times (-0.2) \times \phi'(4.2) \times y_A \\ &= 2 + 0.01 \times (-0.2) \times 2 \\ &= 1.996 \end{aligned}$$

- (c) (6 points) Use backpropagation (and your answers from part (b)) to update the weight w_{1A} . Show your work.

The update for w_{1A} is:

$$\begin{aligned}
 w_{1A} &= 1 - \eta \left(\frac{\partial J}{\partial e_1} \frac{\partial e_1}{\partial y_C} \frac{\partial y_C}{\partial v_C} \frac{\partial v_C}{\partial y_A} + \frac{\partial J}{\partial e_2} \frac{\partial e_2}{\partial y_D} \frac{\partial y_D}{\partial v_D} \frac{\partial v_D}{\partial y_A} \right) \frac{\partial y_A}{\partial w_{1A}} \\
 &= 1 - 0.01 (e_1 \times (-1) \times \phi'(v_C) \times w_{AC} + e_2 \times (-1) \times \phi'(v_D) \times w_{AD}) x_1 \\
 &= 1 + 0.01 (-0.15 \times 0.997 - 0.2 \times 1.996) \times 2 \\
 &= 0.989025
 \end{aligned}$$

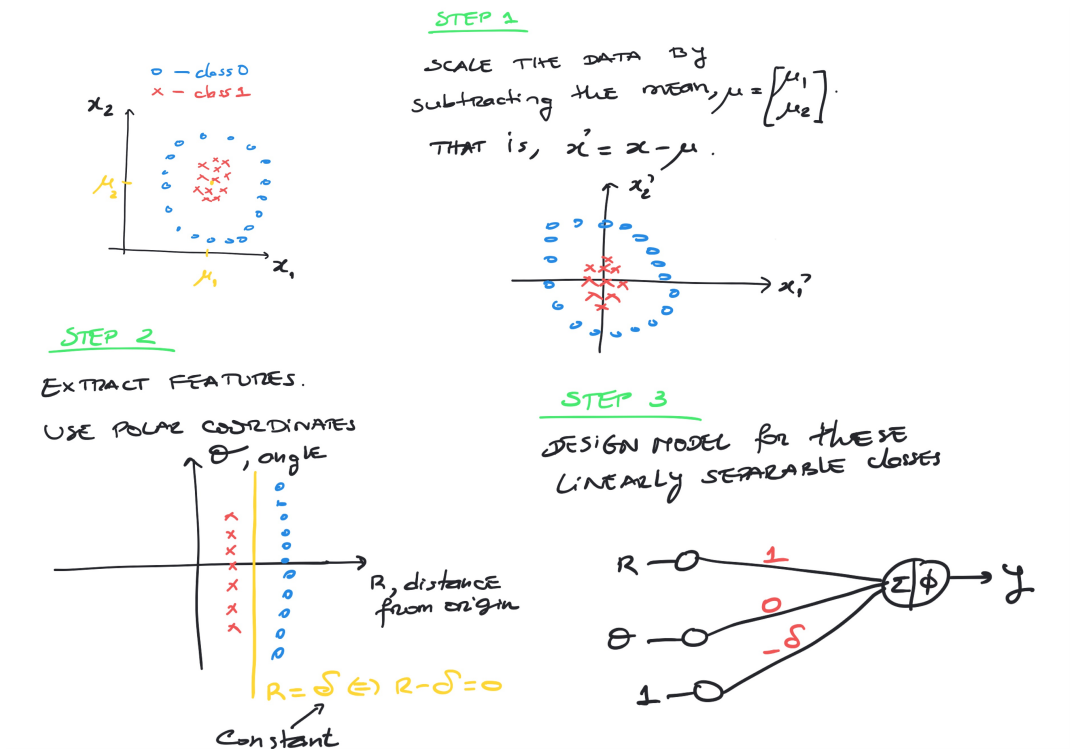
8. (10 points) Consider the following two-dimensional, two-class data set: Design



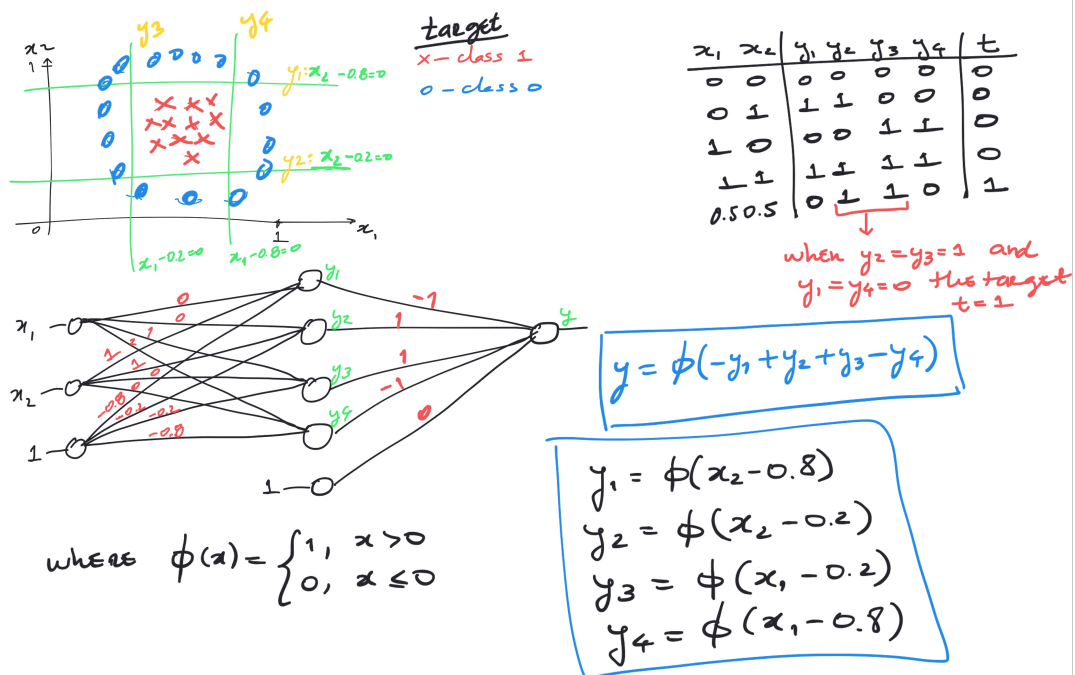
an MLP to obtain 100% classification accuracy on *this* data set. Be sure to define the network architecture and all parameters precisely (give exact numbers). Use the threshold activation function, $\phi(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$, for all units. Show and explain your work.

Multiple answers will be considered correct for this problem.

One way you can approach this problem is to perform a feature transformation, by representing the data in the polar coordinate space and then design an MLP (Perceptron will be sufficient here) in that space.



Alternatively, you can approach this problem without feature transformation:



9. (10 points) **This problem addresses the Multi-Layer Perceptron (MLP). Answer the following questions:**

- (a) (4 points) **How would you encode the output layer, its activation function, and which objective function would you choose to perform the following tasks: binary classification, multi-class classification, single-output regression and multiple-output regression?**

For binary classification, a single unit in the output with a sigmoid activation. This would predict a probability for assigning to class 1. The objective function would be the binary cross-entropy.

For k -class classification, the output layer would have k units with the softmax activation function. Each unit would predict the probability of assigning class $j \in \{1, \dots, k\}$. The objective function would be the multi-class cross-entropy.

For single-output regression, a single unit in the output layer is needed. The activation function can be the linear function. If the target is always positive, then ReLU would also be a good choice. An error-based objective function, like MSE, would be an appropriate objective function.

For multiple-output regression, the output layer would have as many units as there are target variables. The activation function can be the linear function. If the target is always positive, then ReLU would also be a good choice. An error-based objective function, like MSE, would be an appropriate objective function.

- (b) (2 points) **Consider the feature map that results from convolving a $D \times D \times 3$ image a 3×3 filter. How many weight connections are there in this convolutional layer?**

The number of learnable parameter in a convolution layer corresponds to the number of entries in the convolution filter. Since it's a 3×3 , we have a total of 9 learnable parameters (or weight connections). If we include the bias term, then we have a total of $9+1=10$ learnable parameters.

- (c) (4 points) **Identify at least two strategies to mitigate the occurrence of exploding gradients, and at least two strategies to avoid overfitting when training ANNs. Justify your choices.**

Strategies to mitigate the exploding gradients problem include:

- Weight initialization: proper weight initialization allows for the variance of the output of each layer equal to the variances of its inputs.
- Batch normalization: learns the optimal scale and mean of each of the layer's inputs.
- Gradient clipping.
- Regularization: constraint weight and bias connections with L1 or L2 regularizers.

Strategies to avoid overfitting include:

- Weight initialization.
- Batch normalization.

- Regularization.
- Dropout.
- Early stopping.